

SolarPlan NG — Tech Stack Explained

In plain English, for a non-technical reader · 16 September 2026

This document explains **what** was used to build SolarPlan NG, **why** each piece was chosen over the popular alternative, and defines every technical term in plain English. It's written so someone with no coding background can follow it, and specific enough that a developer picking up the project (e.g. in Antigravity IDE) has the real reasoning too.

1. The 30-second version

SolarPlan NG is one website (built with a tool called **Next.js**) that also powers a mobile app and a WordPress plugin, all sharing the exact same "brain" — the solar-sizing math never gets written twice. It stores its data in a database called **PostgreSQL**, hosted on a service called **Neon**. The website itself is hosted on **Vercel**. Payments go through **Paystack**, a Nigerian payment processor. All of the code is written in a language called **TypeScript**, which is JavaScript with extra safety checks built in.

Think of it like a restaurant: Next.js is the whole restaurant building and service system, PostgreSQL/Neon is the pantry where every ingredient (data) is stored and organized, Vercel is the landlord who keeps the lights on and the doors open to customers, Paystack is the card machine at the till, and TypeScript is the recipe book written in a format that catches mistakes (like "add 2 cups of flour" instead of accidentally writing "add 2 cups of flowers") before the mistake ever reaches a customer.

2. The full stack, and why each piece won over its rivals

2.1 Next.js (the web framework) — chosen over plain React, Remix, or a classic PHP site

What it is: A framework is a pre-built skeleton for a website — like buying a house frame instead of cutting every plank of wood yourself. Next.js is one of the most popular frameworks for building modern websites with the React library.

Why Next.js and not the alternatives:

- **vs. plain React:** React alone only handles what happens inside the browser. Next.js adds everything a real business website needs around that — page routing, search-engine-friendly rendering, image handling, and a place to write server-side code (like "check this password" or "charge this card") that never gets sent to the visitor's browser where it could be read or tampered with.
- **vs. Remix or SvelteKit:** Good alternatives, but Next.js has by far the largest ecosystem and community, which matters for a project meant to also power a WordPress plugin and mobile app later — more shared knowledge, more pre-built tools, fewer dead ends.
- **vs. a classic PHP website:** PHP sites (the old WordPress-style approach) mix code and design together in a way that becomes hard to maintain and doesn't share logic cleanly with a mobile app. Next.js keeps the calculation "brain" as reusable code that a website, a mobile app, and a WordPress plugin can all call into — instead of writing the solar math three separate times in three different languages (which is exactly how bugs and inconsistent numbers creep in).

A specific real decision: Midway through, Next.js released version 16 — genuinely newer than what most AI training data knows about, with real breaking changes (e.g., the security-checkpoint file that used to be called `middleware.ts` is now `proxy.ts`). The project used the actual bundled documentation shipped inside Next.js's own files to write correct code for this newer version, rather than guessing from outdated habits.

2.2 TypeScript — chosen over plain JavaScript

What it is: JavaScript is the language every website ultimately runs on. TypeScript is JavaScript with an added layer that checks your work before it ever runs — like a spellchecker, but for logic. If a piece of code expects a number and you accidentally hand it a word, TypeScript flags it immediately while writing the code, instead of the website crashing later for a real visitor.

Why it matters here specifically: SolarPlan NG's entire value proposition is "these numbers are computed correctly, never guessed." A calculation engine with actual type-safety mistakes (say, treating "6" the number as "6" the text) is exactly the kind of subtle bug that could quietly hand someone the wrong inverter size. TypeScript catches an entire category of these mistakes automatically.

2.3 PostgreSQL (via Neon) — chosen over MySQL or MongoDB

What it is: A database is where all the persistent information lives — every user's saved calculation, every product, every payment record. PostgreSQL is one of the two most popular "relational" databases (the other being MySQL); Neon is a company that hosts PostgreSQL for you in the cloud, so nobody has to manage a physical server.

- **vs. MySQL:** Both are strong choices; PostgreSQL was picked for its stronger support for the kind of relationship-heavy, data-integrity-critical records this app needs (a Payment must always correctly link to exactly one Report, a Product must always correctly link to one Brand, etc.) and for future-proofing — it supports advanced location-based queries (useful for "find the nearest installer") that MySQL handles less natively.
- **vs. MongoDB (a "NoSQL" database):** NoSQL databases store loosely-structured, flexible records — good for content that doesn't follow a strict shape. SolarPlan NG's data (payments, products, verified appliance specs) is the opposite: it needs strict structure and guaranteed relationships so a payment can never accidentally end up detached from what was paid for. A relational database enforces that by design; a NoSQL one would need that same discipline bolted on manually, with more room for error.
- **Why Neon specifically, as the host:** Neon offers a genuinely free starter tier and — notably — lets the database "sleep" and wake instantly, and supports cheaply duplicating (branching) the whole database for testing, which suits a project still in active development.

2.4 Prisma (the ORM) — chosen over writing raw database queries, or a newer tool called Drizzle

What it is: An ORM ("Object-Relational Mapper") is a translator between the code and the database. Instead of writing raw database instructions by hand (a language called SQL), the code says something like "give me this Product" in plain code, and Prisma translates that into the correct, safe database query behind the scenes.

- **vs. writing raw SQL by hand:** Faster to build with, and it automatically prevents an entire class of security bug called "SQL injection" (where a malicious visitor tries to sneak database commands into a text box). It also auto-generates a strongly-typed "menu" of every table and field, so TypeScript can catch a typo like `procut` instead of `product` before the code ever runs.
- **vs. Drizzle (a newer, lighter alternative):** Drizzle is a legitimate, more lightweight competitor. Prisma was picked here for its more mature migration system (the tool that safely evolves the database's structure over time as new features are added) and its stronger day-to-day developer tooling.

A specific real decision (and a real trap avoided): Prisma released a major new version — "Prisma 7" — partway through, and simply asking for "the latest version" (a common instinct) actually installs an entirely different, unrelated next-generation rewrite called "Prisma Composer" (labelled version 8), which has no Windows support for local development and works completely differently. This was caught early, and the project was deliberately pinned to the real, stable Prisma 7 instead.

2.5 Tailwind CSS — chosen over Bootstrap or hand-written CSS

What it is: CSS is the language that controls how a website looks (colors, spacing, fonts). Tailwind is a popular way of writing CSS using small, reusable "utility" labels directly in the code (e.g. `text-lg` means "large text") instead of writing separate style sheets by hand.

- **vs. Bootstrap:** Bootstrap ships pre-designed components that immediately look like every other "Bootstrap site" unless heavily customized. Tailwind is closer to raw building blocks, which is why SolarPlan NG could get a genuinely distinctive visual identity (the amber "sun" theme) rather than a generic, templated look.
- **vs. hand-written CSS:** Hand-written CSS is completely flexible but slow, and it's easy for styles to drift out of sync across dozens of pages. Tailwind keeps styling consistent and fast to change everywhere at once.

2.6 Vercel — chosen over a traditional cloud server (AWS, DigitalOcean) or Netlify

What it is: Hosting is the service that actually puts a website on the internet and keeps it running. Vercel is a hosting company built specifically around Next.js (the same company that makes Next.js).

- **vs. a traditional server (AWS EC2, DigitalOcean):** Those require manually configuring and maintaining a server — installing software, managing security patches, setting up scaling. Vercel handles all of that automatically and deploys a new version of the site every time the code changes, with instant one-click rollback if something breaks.
- **vs. Netlify:** A very similar, reputable competitor — but since the site is built with Next.js, Vercel (Next.js's own creator) has the tightest, most reliable integration and best support for Next.js-specific features.
- **A practical factor:** The project owner already had a Vercel account set up and ready before the project began, which made it the immediate practical choice too.

2.7 Paystack — chosen over Flutterwave or Stripe

What it is: A payment processor is the service that actually handles a customer's card details and moves money — no website should ever handle raw card numbers itself. Paystack is one of the leading payment processors built specifically for Nigerian and African businesses.

- **vs. Stripe:** Stripe is the global leader, but historically has had a much rockier, less direct path for Nigerian businesses to receive payouts in naira. Paystack is built around Nigerian banking rails from the ground up.
- **vs. Flutterwave:** Another strong, popular Nigerian alternative. Paystack was the owner's explicit choice; the payment code was deliberately written behind a clean internal boundary so a second processor like Flutterwave could technically be added later without rewriting the checkout flow.

2.8 Vitest (testing) — chosen over Jest

What it is: A testing tool automatically checks that the code still behaves correctly every time something changes — for example, re-running the exact solar-sizing formulas against known correct answers to make sure nobody accidentally broke the math while adding a new feature.

Why Vitest and not Jest (the older, more famous option): Vitest is built to work natively with the same modern build tooling Next.js already uses, so tests run noticeably faster with less configuration friction, while working almost identically to Jest for anyone already familiar with it.

2.9 Zod (validation) — the safety net at every door

What it is: Zod checks that data coming into the system (a form submission, an API request) actually matches the shape it's supposed to — the right fields, the right types, the right limits — before any of it is trusted or used.

Why it matters here: Every public entry point (the checkout form, the invoice creator, the public API used by the mobile app and WordPress plugin) runs incoming data through a Zod check first. This is what stops, for example, someone submitting a 10,000-line fake invoice to try to overload the system, or a malformed calculation request crashing the server.

3. Glossary — every technical term, in plain English

TERM	PLAIN-ENGLISH MEANING
npm	"Node Package Manager." Think of it as an app store, but for code — it downloads and manages the thousands of small, reusable pieces of code (called packages) a project depends on, instead of everyone writing everything completely from scratch.
npmx	A close cousin of npm that runs a tool once without permanently installing it — like using a tool at a hardware store's rental counter instead of buying it.
Node.js	The engine that lets JavaScript code run outside of a web browser — for example, on a server, or on a developer's own computer. Almost everything in this project runs on top of it.
package.json	The project's master "ingredients list" — a file naming every external piece of code (package) the project depends on, and the exact version of each.
Package / dependency	A ready-made, reusable chunk of code someone else wrote and shared, that this project uses instead of reinventing (e.g., the code that generates a PDF file).
Framework	A pre-built skeleton/structure for building software, with strong opinions about how things should be organized (Next.js is a framework).
Library	A more flexible, "grab what you need" toolbox of reusable code, with fewer built-in rules about structure (React is technically a library).
Repository ("repo")	The complete folder of a project's code, tracked over time so every past version is recoverable.
Git	The specific tool that tracks every change to the code over time — like a very detailed, permanent "track changes" history for the whole project.
Commit	One saved snapshot of the code at a specific point in time, with a short note explaining what changed and why.
Deploy / deployment	The act of publishing a new version of the code so it becomes the live, real website visitors actually see.

Build	The automated process that takes the raw source code and turns it into the optimized, ready-to-run version that actually gets deployed.
API	A defined set of "doors" that let separate pieces of software talk to each other — e.g., the mobile app doesn't reimplement the solar math itself; it just asks the API, "given these appliances, what size inverter do I need?" and gets back an answer.
Endpoint	One specific, named "door" in an API — e.g. the endpoint that calculates a solar system size is different from the endpoint that looks up Nigerian states.
JSON	A simple, universal text format for exchanging structured data between different pieces of software (e.g., <code>{"watts": 100}</code>) — readable by both humans and computers.
Database	The organized, permanent storage system for all of the app's real information (users, products, payments) — separate from the website's code itself.
Schema	The database's official "blueprint" — the definition of exactly what tables and fields exist, and how they relate to each other.
Migration	A tracked, reversible change to the database's structure over time (e.g., "add a new phone number field to every vendor") — done carefully so existing data is never accidentally lost.
ORM	A translator layer between the code and the database (see Prisma, §2.4) that lets the code ask for data in plain language instead of writing raw database instructions.
SQL	The specific language databases like PostgreSQL actually understand under the hood (Prisma writes this automatically on the code's behalf).
Server	The remote computer that runs the website's back-end code and serves pages to visitors — as opposed to code that runs directly in a visitor's own browser.
Server-side vs. client-side	"Server-side" code runs privately on the hosting company's computer (safe for secrets, like checking a password); "client-side" code runs on the visitor's own device inside their browser (visible to them, so never trusted with secrets).
Environment variable	A setting or secret (like a database password or API key) kept outside the actual code, so it can differ between a developer's own computer and the real live website, and so secrets never get accidentally published.
.env file	The specific file on a developer's own computer that holds those environment variables for local testing — deliberately never saved into the shared repository.
CLI	A tool controlled by typing text commands (like <code>npx vercel deploy</code>) rather than clicking buttons in an app.
Token / API key / secret key	A private, password-like string that proves a request is authorized to use a service — e.g. Paystack's secret key proves the website is really allowed to process a payment on its behalf. Never meant to be shared publicly.
Webhook	A message one service automatically sends to another the instant something happens — e.g., Paystack "pings" SolarPlan NG's server the moment a payment succeeds, rather than SolarPlan NG having to constantly ask "has it been paid yet?"

Cookie / session	A small piece of information a website asks a visitor's browser to remember, commonly used to keep someone signed in between page visits.
Rate limiting	A safety rule that caps how many requests one visitor can make in a short time, to prevent abuse or accidental overload (e.g., "max 5 checkout attempts per minute").
Responsive design	Building a page so it automatically rearranges and resizes correctly on any screen — a phone, a tablet, a desktop monitor.
Accessibility (a11y)	Designing a site so it also works properly for people using screen readers, keyboard-only navigation, or who have visual impairments (e.g., text color must stay readable enough against its background).
SEO	The practice of structuring a site so Google and other search engines can find, understand, and rank its pages correctly.
Sitemap / robots.txt	Two small files that tell search engines which pages exist and are safe to show in search results, and which pages (like a private payment receipt link) should stay out of search results entirely.
HTTPS / SSL	The padlock-icon encryption that keeps data private between a visitor's browser and the website — standard and automatic on Vercel.
DNS	The internet's "phone book" — the system that turns a typed domain name (like solarplanng.com) into the actual technical address of the server hosting it.
CDN	A network of servers spread around the world that serve a website's files from whichever copy is physically closest to each visitor, making the site load faster everywhere.
Staging vs. production	"Production" is the real, live website the public sees. "Staging" (not currently used in this project, but common practice) is a private copy used to test changes safely before they go live.
Unit test	A small, automated check that one specific piece of code still does exactly what it's supposed to — run automatically, catching mistakes before they ever reach real visitors.
Linting	An automated tool that checks code for style mistakes, common bugs, and inconsistencies — like a grammar-checker for code.
TypeScript / type-checking	See §2.2 — an added safety layer on top of JavaScript that catches a whole category of mistakes before code ever runs.